

configevaluator

Security Test Report

VERSION	TEST DATE	RESULT
0.1.0	14 July 2026	ALL TESTS PASSED

1. Purpose and scope

This report verifies that **configevaluator** operates as an offline security linter for `.env` configuration files. The tested CLI identifies common insecure settings, explains remediation, validates malformed input, supports structured JSON output, and supplies automation-friendly exit statuses.

Testing used only fictional local fixtures from the repository. No public hosts were scanned, no credentials were tested, and no configuration values were transmitted.

2. Test environment

Item	Tested value
Operating system	macOS 26.3.1 (Build 25D771280a)
Architecture	Apple arm64
Python	Python 3.13.9
Tool version	0.1.0
Execution mode	Local source package via PYTHONPATH=src
Commit identifier	Not used; tool version recorded instead

VERDICT	The implementation satisfied every expected result in this report: 7 of 7 report cases passed, and the complete automated suite passed 12 of 12 tests.
---------	--

3. Test cases

Each case was executed from the repository root. Expected and actual outcomes were compared directly; status reflects the observed exit status and output.

ID	Command / input	Expected result	Actual result	Status
T1	<code>python3 -m unittest discover -v</code>	All automated tests pass; exit 0	12 tests passed; exit 0	PASS
T2	<code>configevaluator examples/insecure.env</code>	Detect 4 high and 4 medium findings	4 high, 4 medium, 0 low; exit 0	PASS
T3	<code>configevaluator examples/secure.env</code>	No known insecure settings	Clean advisory shown; exit 0	PASS
T4	<code>configevaluator examples/malformed.env</code>	Line-specific format error; exit 2	Line 2 expected KEY=value; exit 2	PASS
T5	<code>configevaluator examples/does-not-exist.env</code>	Clear missing-file error; exit 2	File does not exist; exit 2	PASS
T6	<code>configevaluator examples/insecure.env --format json</code>	Valid JSON with accurate summary	Valid JSON; summary 4/4/0; exit 0	PASS
T7	<code>configevaluator examples/insecure.env --fail-on high</code>	Findings shown; exit 1	High finding threshold met; exit 1	PASS

Coverage summary

- Normal behavior: secure and intentionally insecure configuration files.
- Invalid behavior: malformed syntax and a missing input path.
- Edge and integration behavior: duplicates, quoted values, values containing equals signs, local HTTP exceptions, severity filtering, JSON serialization, secret-safe output, and exit thresholds.

4. Terminal capture - automated tests

The complete built-in test suite exercises the parser, all primary rule behavior, CLI output, JSON validity, missing-file handling, severity filters, and failure thresholds.

AUTOMATED TEST SUITE

```
$ PYTHONPATH=src python3 -m unittest discover -v
test_fail_on_threshold_returns_one (tests.test_cli.CliTests) ... ok
test_json_output_is_valid_and_does_not_expose_secret (tests.test_cli.CliTests) ... ok
test_missing_file_returns_two (tests.test_cli.CliTests) ... ok
test_severity_filter_hides_lower_findings (tests.test_cli.CliTests) ... ok
test_parses_comments_quotes_and_equals_in_values (tests.test_parser.ParserTests) ... ok
test_rejects_malformed_line_with_line_number (tests.test_parser.ParserTests) ... ok
test_rejects_unmatched_quote (tests.test_parser.ParserTests) ... ok
test_retains_empty_values_and_duplicate_keys (tests.test_parser.ParserTests) ... ok
test_detects_all_primary_insecure_patterns (tests.test_rules.RuleTests) ... ok
test_duplicate_key_reports_later_occurrence (tests.test_rules.RuleTests) ... ok
test_local_http_url_is_allowed (tests.test_rules.RuleTests) ... ok
test_secure_settings_have_no_findings (tests.test_rules.RuleTests) ... ok

-----
Ran 12 tests in 0.003s

OK
$ exit status: 0
```

Figure 1. Actual automated test output captured on 14 July 2026.

5. Insecure input evidence

The intentionally insecure fixture demonstrates severity ordering, stable rule identifiers, input line numbers, explanations, and actionable remediation without printing configuration values.

```
INSECURE CONFIGURATION SCAN

$ PYTHONPATH=src python3 -m configevaluator examples/insecure.env
ConfigEvaluator scan
File: examples/insecure.env

HIGH   CFG008   line 2   HOST
        Debug mode may be exposed on a public network interface.
        Recommendation: Disable debug mode or bind the application to a local interface.

HIGH   CFG002   line 3   SECRET_KEY
        A placeholder secret value was detected.
        Recommendation: Replace it with a securely generated secret.

HIGH   CFG003   line 4   DATABASE_PASSWORD
        A sensitive setting has an empty value.
        Recommendation: Provide the value securely at deployment time.

HIGH   CFG007   line 6   VERIFY_TLS
        TLS certificate verification is disabled.
        Recommendation: Enable certificate verification and use a trusted CA.

MEDIUM CFG001   line 1   DEBUG
        Debug mode is enabled.
        Recommendation: Disable debug mode in production.

MEDIUM CFG005   line 5   CORS_ALLOW_ORIGINS
        Cross-origin access is allowed from every origin.
        Recommendation: List only the trusted origins that require access.

MEDIUM CFG006   line 7   SESSION_COOKIE_SECURE
        Secure cookies are disabled.
        Recommendation: Enable secure cookies when the application uses HTTPS.

MEDIUM CFG004   line 8   API_URL
        A service URL uses unencrypted HTTP.
        Recommendation: Use HTTPS when communicating beyond the local machine.

Summary: 4 high, 4 medium, 0 low
$ exit status: 0
```

Figure 2. Actual scan output for examples/insecure.env.

6. Clean and invalid input evidence

This capture verifies a clean result, two user-facing error paths, and the nonzero status used when high-severity findings should fail an automated check.

CLEAN INPUT, ERRORS, AND FAILURE THRESHOLD

```
$ PYTHONPATH=src python3 -m configevaluator examples/secure.env
ConfigEvaluator scan
File: examples/secure.env

No known insecure settings detected.
Note: this result does not guarantee that the configuration is secure.
$ exit status: 0

$ PYTHONPATH=src python3 -m configevaluator examples/malformed.env
configevaluator: invalid input: line 2: expected KEY=value
$ exit status: 2

$ PYTHONPATH=src python3 -m configevaluator examples/does-not-exist.env
configevaluator: cannot read examples/does-not-exist.env: file does not exist
$ exit status: 2

$ PYTHONPATH=src python3 -m configevaluator examples/insecure.env --fail-on high
[standard findings omitted from this capture]
$ exit status: 1
```

Figure 3. Actual clean-input, malformed-input, missing-file, and threshold results.

7. Known limitations

Version 0.1.0 supports a deliberately limited subset of `.env` syntax. Inline comments, variable expansion, multiline values, and the `export` prefix are not parsed. Rules recognize common configuration key names, so framework-specific settings may be missed and context-dependent settings may produce false positives. A clean report is therefore not proof that a configuration is secure. These limitations are documented in the README and user manual.

No unresolved functional bug was observed in the tested cases.

8. Conclusion

configevaluator completed all seven report cases successfully, while its complete automated suite passed all twelve tests. The tested release meets its intended defensive purpose: safe, local analysis of common `.env` configuration risks with clear remediation and predictable error handling.